



Praktikum Pemrograman Mobile

Pertemuan 4:

## ***Recomposition dan UI Lifecycle***

Disusun oleh :

**Azis Amirulbahar, S.P.d., M.T.I.**

**Program Studi Informatika**

**Fakultas Teknik**

**Universitas Jenderal Soedirman**

**2026**



# DAFTAR ISI

|                                                                               |          |
|-------------------------------------------------------------------------------|----------|
| <b>DAFTAR ISI</b> .....                                                       | <b>2</b> |
| <b>PENDAHULUAN</b> .....                                                      | <b>3</b> |
| A. Pengantar .....                                                            | 3        |
| B. Tujuan Praktikum .....                                                     | 3        |
| C. Alat dan Bahan .....                                                       | 4        |
| D. Dasar Teori.....                                                           | 4        |
| <b>TAHAPAN PRAKTIKUM</b> .....                                                | <b>5</b> |
| A. Modifikasi <i>Permissions</i> .....                                        | 5        |
| B. Modifikasi Form Hubungi Kami .....                                         | 6        |
| C. Modifikasi kode pada <b>DaftarProductScreen.kt</b> .....                   | 17       |
| D. Membuat Tampilan Detail Produk .....                                       | 23       |
| E. Navigasi di <i>homeactivity</i> untuk daftar produk ke detail produk ..... | 27       |
| F. Menambahkan Menu Action di AppBar untuk navigasi ke hubungi kami .....     | 28       |

# PENDAHULUAN

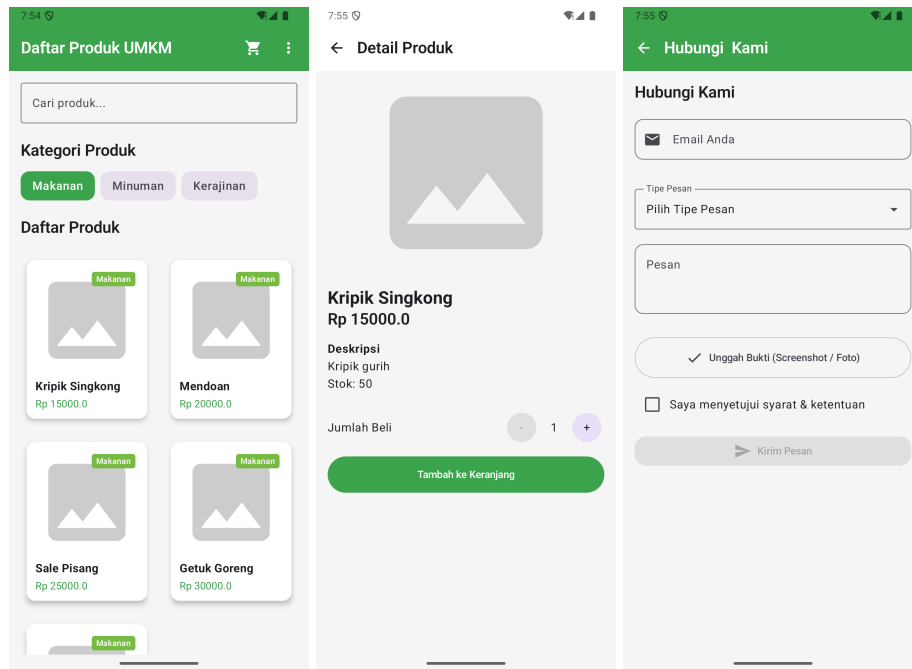
## A. Pengantar

Pada pertemuan ke-4 ini, kita akan mempelajari konsep lanjutan dari Jetpack Compose yaitu pengelolaan *State*, *Recomposition*, *State Hoisting*, dan simulasi proses Asinkronus. Pada pertemuan sebelumnya, kita telah berhasil membuat layout dinamis menggunakan *LazyList*. Namun, antarmuka aplikasi yang baik tidak hanya menampilkan data statis, melainkan harus interaktif dan responsif terhadap aksi pengguna seperti mengetik pada form atau mencari sebuah data.

**“Modul pada pertemuan ke-4 dibuat lebih ringkas daripada pertemuan – pertemuan sebelumnya. Peserta diharapkan dapat secara mandiri untuk melakukan implementasi suatu tampilan dari kode yang dibuat dengan mengadopsi pertemuan sebelumnya, tanpa petunjuk/panduan baris kode seperti pada pertemuan sebelumnya Terdapat pula implementasi komponen yang sama dengan pertemuan – pertemuan sebelumnya, sehingga penjelasan kode tidak perlu disertakan pada pertemuan ini dan peserta diharapkan secara mandiri untuk dapat mengimplementasikannya.”**

## B. Tujuan Praktikum

1. Memahami alur kerja *Recomposition* dan *UI Lifecycle* dalam antarmuka deklaratif.
2. Menerapkan *State Hoisting* untuk memisahkan *Stateful Composable* (pengelola data) dan *Stateless Composable* (penampil UI).
3. Menggunakan konsep *Kotlin Lambda* dan *Higher-Order Functions* sebagai jembatan *event* / aksi pengguna.
4. Membuat antarmuka form kompleks interaktif (Dropdown/Combobox, Checkbox, Image Upload) beserta validasinya.
5. Memahami dan menerapkan proses asinkronus (Coroutine) dasar menggunakan *LaunchedEffect* di Compose untuk memunculkan indikator *Loading*.



Gambar 1. Hasil Akhir Praktikum

## C. Alat dan Bahan

1. Perangkat keras (PC/Laptop) dengan spesifikasi minimum sesuai standar Android Studio.
2. Android Studio (versi terbaru yang sudah mendukung Jetpack Compose secara bawaan).
3. Emulator Android (AVD) atau perangkat fisik Android yang terhubung melalui *debugging* (USB/Wi-Fi).

## D. Dasar Teori

1. **State dan Recomposition:** *State* adalah data yang dapat berubah seiring waktu dan menentukan apa yang dirender di layar (misal: teks yang sedang diketik user di kolom pencarian). *Recomposition* adalah proses otomatis di mana Compose memanggil ulang (menggambar ulang) bagian fungsi UI untuk memperbarui tampilan layar saat nilai *State* tersebut berubah.
2. **Unidirectional Data Flow (UDF):** Pola desain standar di arsitektur modern di mana data (*State*) selalu mengalir turun dari fungsi induk (Parent) ke fungsi anak (Child), sedangkan aksi (*Event*) seperti klik selalu mengalir naik dari fungsi anak ke fungsi induk menggunakan fungsi *Lambda*.

3. **State Hoisting:** Teknik memindahkan pengelolaan variabel *State* ke fungsi induk (*Stateful Composable*) dan membiarkan fungsi UI di bawahnya murni hanya menerima data tanpa menyimpan data sendiri (*Stateless Composable*). Hal ini membuat fungsi UI lebih mudah diuji, tidak rawan *bug*, dan dapat digunakan kembali (*reusable*).
4. **LaunchedEffect & Coroutine:** Lingkungan khusus di dalam Compose yang memungkinkan eksekusi kode *Asynchronous* (berjalan di *background thread*). Fungsi ini sangat esensial untuk operasi yang memakan waktu, seperti mengunduh data dari API Server atau simulasi jeda sistem, tanpa menyebabkan UI utama macet atau berhenti merespons.

## TAHAPAN PRAKTIKUM

Pada bagian ini, peserta diminta untuk mengikuti tahapan – tahapan sehingga menghasilkan aplikasi sederhana seperti pada gambar 1 di halaman sebelumnya.

### A. Modifikasi *Permissions*

*Android Permission (Hak Akses)* adalah mekanisme keamanan utama pada sistem operasi Android yang dirancang untuk melindungi privasi pengguna. Secara bawaan (default), sebuah aplikasi Android berjalan di dalam “kotak pasir” (*sandbox*) terisolasi dan sama sekali tidak memiliki akses untuk menggunakan sumber daya atau data sensitif pada perangkat pengguna (seperti kamera, daftar kontak, lokasi GPS, hingga memori penyimpanan file).

Agar aplikasi dapat mengakses sumber daya luar tersebut (dalam praktikum ini: fitur Unggah Gambar yang mengharuskan aplikasi masuk dan membaca galeri foto pengguna), aplikasi diwajibkan untuk mendeklarasikan permintaan izin secara eksplisit di dalam sebuah file konfigurasi utama yang bernama **AndroidManifest.xml**. Tanpa adanya deklarasi izin ini, sistem operasi Android akan memblokir paksa upaya aplikasi saat mencoba membuka memori HP, yang dapat menyebabkan aplikasi crash (*Force Close*).

1. Silakan Buka File AndroidManifest.xml yang terletak di app/src/main/AndroidManifest.xml. Kemudian tambahkan kode berikut di atas TAG <application>

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />

    <application
```

Gambar 2. Modifikasi AndroidManifest.xml

## B. Modifikasi Form Hubungi Kami

Pada bagian ini, kita akan melakukan modifikasi form hubungi kami dengan menerapkan *lambda expression*, *state management*, serta implementasi permissions untuk membuka gambar dari galeri.

1. Tambahkan deklarasi variabel di dalam function HubungiKamiScreen sesuai dengan gambar berikut dan lakukan import (atau tekan tombol Control+Space) jika kode terdapat error yang ditandai dengan warna merah.

```
fun HubungiKamiScreen(navController: NavController?) {
    var emailText by remember { mutableStateOf( value = "" ) }
    var messageText by remember { mutableStateOf( value = "" ) }
    var problemType by rememberSaveable { mutableStateOf( value = "Pilih Tipe Pesan" ) }
    var isAgreed by rememberSaveable { mutableStateOf( value = false ) }
    var imageUri by remember { mutableStateOf<Uri?>( value = null ) }
```

Gambar 3. Deklarasi Variabel

Penjelasan kode:

Kode pada gambar 3 merupakan penerapan tiga konsep pada ekosistem Jetpack Compose dan Kotlin yaitu: konsep **State & Recomposition**, penggunaan fungsi remember dan rememberSaveable, serta fitur pada Kotlin bernama **Property Delegation**.

fungsi mutableStateOf(...) Dalam paradigma UI deklaratif Jetpack Compose, akan membuat tampilan layar bersifat reaktif terhadap perubahan data. Tipe data primitif biasa seperti String atau Boolean tidak mampu memberi

sinyal ke Compose jika nilainya berubah. Oleh karena itu, perlu “membungkus” data tersebut ke dalam `mutableStateOf`. Sehingga, setiap kali nilai di dalam objek tersebut diperbarui, maka Jetpack Compose akan mendeteksi perubahan tersebut secara otomatis dan memicu proses ***recomposition***, yaitu proses menggambar ulang bagian-bagian layar yang bergantung pada variabel tersebut agar selalu menampilkan data yang *ter-update*.

Namun, jika hanya menuliskan `mutableStateOf("")` tanpa pembungkus, nilai variabel akan selalu kembali ke nilai awal saat deklarasi setiap kali layar “digambar ulang”. Sehingga diperlukan fungsi `Remember`, yang akan memerintahkan Jetpack Compose untuk menyimpan dan “mengingat” nilai variabel tersebut di dalam *composition tree* selama siklus hidup komponen berlangsung. Namun, `remember` memiliki Batasan yaitu data akan hilang jika terjadi perubahan konfigurasi perangkat (*configuration change*), seperti saat layar ponsel diputar dari mode *portrait* ke *landscape* atau sebaliknya. Untuk mengatasi hal itu, diperlukan `rememberSaveable`, seperti yang telah dideklarasikan pada variabel `problemType` dan `isAgreed`. Fungsi `rememberSaveable` akan menyimpan data ke dalam mekanisme `SavedInstanceState` Android, sehingga data pilihan dan centang pengguna tetap aman dan tidak terreset meskipun orientasi layar berubah atau proses aplikasi sempat dihentikan sementara oleh sistem operasi.

**Property Delegation** (delegasi properti) dengan kata kunci `by` akan “menyamarkan” kompleksitas pembungkus tersebut. Sehingga, kita bisa melakukan *set* nilai dalam variabel tanpa perlu berulang kali mengetik `.value`. Hal serupa juga diterapkan pada `imageUri` dengan anotasi tipe generik `mutableStateOf<Uri?>(null)`, yang secara eksplisit memberitahu Kotlin bahwa variabel tersebut memegang rujukan lokasi berkas gambar yang boleh bernilai kosong (*nullable*).

- Setelah mendeklarasikan variabel sesuai dengan gambar 3, tambahkan variabel untuk menyimpan status validasi:

```
val isValidEmail = emailText.contains( other = "@" ) && emailText.isNotBlank()  
val isValidMessage = messageText.length >= 10  
val isValidForm = isValidEmail && isValidMessage && isAgreed && problemType != "Pilih Tipe Pesan"  
  
val scope = rememberCoroutineScope()  
val snackbarHostState = remember { SnackbarHostState() }
```

Gambar 4. Deklarasi Variabel

#### Penjelasan Kode:

Variabel `isValidEmail` dievaluasi menggunakan operator logika `&&` (AND) dan fungsi bawaan (*standard library*) Kotlin, yaitu `contains("@")` serta `isNotBlank()`. Fungsi `isNotBlank()` memastikan bahwa teks email tidak sekadar berupa deretan spasi kosong, sekaligus melengkapi pengecekan keberadaan karakter "@". Pada baris berikutnya, `isValidMessage` menggunakan properti bawaan `length` pada `String` untuk memastikan panjang karakter pesan bernilai minimal 10 karakter (`>= 10`). Kedua ekspresi ini menghasilkan tipe data primitif `Boolean` (`true` atau `false`) secara ringkas tanpa memerlukan blok percabangan `if-else` yang Panjang seperti pada bahasa pemrograman lain.

`isValidForm` merupakan kombinasi dari seluruh aturan validasi formulir. Variabel tersebut menggabungkan hasil validasi sebelumnya (`isValidEmail` dan `isValidMessage`) dengan status persetujuan syarat ketentuan (`isAgreed`) serta memastikan pengguna telah memilih kategori masalah yang sah, yaitu nilainya tidak lagi sama dengan teks awal "Pilih Tipe Pesan" (`!=`). Jika dan hanya jika seluruh kondisi ini terpenuhi, `isValidForm` akan bernilai `true`.

3. Masih dalam file `HubungiKamiScreen.kt`, buatlah deklarasi *function* `StatelessFormHubungiKami()` seperti pada gambar 3 berikut:

```
@Composable
fun StatelessFormHubungiKami(
    modifier: Modifier = Modifier,
    email: String, onEmailChange: (String) -> Unit, isValidEmail: Boolean,
    message: String, onMessageChange: (String) -> Unit, isValidMessage: Boolean,
    problemType: String, onProblemTypeChange: (String) -> Unit,
    isAgreed: Boolean, onAgreedChange: (Boolean) -> Unit,
    imageUri: Uri?, onImagePicked: (Uri?) -> Unit,
    isValidForm: Boolean, onSubmit: () -> Unit
){
}
}
```

Gambar 5. Deklarasi Function `StatelessFormHubungiKami()`

Penjelasan kode:

Kode di atas mendefinisikan sebuah fungsi antarmuka (UI) bernama `StatelessFormHubungiKami`. Dalam paradigma Jetpack Compose, komponen UI idealnya dipisahkan antara yang memegang status (*stateful*) dan yang murni hanya bertugas menggambar tanpa menyimpan data internal (*stateless*). Konsep ini berakar pada teori **State Hoisting**, yaitu pola desain untuk menaikkan kepemilikan data ke komponen induk (*parent*) atau ke lapisan arsitektur yang lebih tinggi seperti `ViewModel` (yang akan dibahas pada pertemuan berikutnya). Manfaatnya, komponen `StatelessFormHubungiKami` menjadi sangat modular, mudah diuji (*testable*), dan bisa digunakan ulang (*reusable*) di berbagai layar tanpa terikat pada sumber data tertentu.

Penerapan *State Hoisting* pada parameter disusun menggunakan pola **Unidirectional Data Flow (UDF)**. Pada sistem deklaratif, **aliran data bergerak satu arah: data mengalir ke bawah menuju komponen UI, sedangkan aksi pengguna mengalir ke atas sebagai event**. Parameter `email` (tipe `String`) dan `onEmailChange: (String) -> Unit` adalah contoh penerapannya. Komponen tersebut menerima teks email untuk ditampilkan ke layar, dan ketika pengguna mengetik karakter baru di papan ketik, fungsi ini tidak mengubah nilai variabel secara langsung. Fungsi tersebut justru memanggil *lambda expression* atau fungsi tingkat tinggi (*higher-order function*) di Kotlin bernama `onEmailChange` untuk memberitahu

ke komponen induk bahwa ada perubahan data. Pola yang sama berulang pada penanganan pesan, tipe kendala, kotak centang persetujuan (`isAgreed`), hingga pemilihan berkas gambar (`imageUri` yang bertipe *nullable Uri*?).

4. Deklarasikan Variabel Untuk `PhotoPicker` di dalam function `StatelessFormHubungiKami()` tersebut seperti pada gambar berikut dan lakukan *import* terhadap *dependency/package* yang dibutuhkan:

```
val photoPickerLauncher = rememberLauncherForActivityResult(  
    contract = ActivityResultContracts.PickVisualMedia(),  
    onResult = { uri -> onImagePicked(uri) }  
)
```

Gambar 6. Deklarasi variabel `PhotoPicker`

Penjelasan kode:

Fungsi `rememberLauncherForActivityResult` bertindak sebagai “jembatan” yang mendaftarkan *launcher* ke sistem Android dan “mengingatnya” agar tidak terbuat ulang saat layar di-refresh (*recomposition*). Kode `contract = ActivityResultContracts.PickVisualMedia()` berfungsi mengakses foto tanpa perlu meminta izin akses memori (*storage permission*). blok `onResult = { uri -> onImagePicked(uri) }` menerima alamat lokasi foto (`Uri`) yang dipilih oleh pengguna, lalu langsung meneruskannya ke fungsi `onImagePicked` untuk memperbarui tampilan layar.

5. Pindahkan seluruh kode *Column* beserta *child*-nya di function *HubungiKamiScreen* ke dalam function *StatelessFormHubungiKami()*

```
Column(
  modifier = Modifier
    .fillMaxSize()
    .padding(paddingValues)
    .padding( all = 16.dp),
  horizontalAlignment = Alignment.CenterHorizontally
) {

  Text(
    text = "Hubungi Kami",
    style = MaterialTheme.typography.titleLarge,
    modifier = Modifier.align(Alignment.Start)
  )

  Spacer(modifier = Modifier.height(16.dp))

  OutlinedTextField(
    value = emailText,
    onValueChange = { emailText = it },
    label = { Text("Email Anda") },
    leadingIcon = {
      Icon(
        painter = painterResource(id = R.drawable.mail_icon),
        contentDescription = "Email"
      )
    },
    modifier = Modifier.fillMaxWidth(),
    shape = MaterialTheme.shapes.medium
  )
}
```

Gambar 7. Blok kode yang perlu dipindah ke function *StatelessFormHubungiKami()*

6. Hapus modifier padding yang memiliki parameter *PaddingValues* pada komponen *Column*

```
Column(
  modifier = Modifier
    .fillMaxSize()
    .padding(paddingValues)
    .padding( all = 16.dp),
  horizontalAlignment = Alignment.CenterHorizontally
) {
```

Gambar 8. Penghapusan baris kode pada komponen *Column*

7. Ganti variabel *modifier* seperti gambar di bawah ini yang mengacu pada parameter *function* (huruf M kecil):

```
Column(  
    modifier = modifier  
        .fillMaxSize()  
        .padding( all = 16.dp),  
    horizontalAlignment = Alignment.CenterHorizontally  
) {
```

Gambar 9. Penggantian parameter

8. Ganti Nilai komponen `OutlinedTextField` pada parameter `onValueChange` dengan parameter `onEmailChange` serta tambahkan Properti `isError` & `SupportingText` pada `outlinedtext` seperti gambar berikut:

```
OutlinedTextField(  
    value = emailText,  
    onValueChange = onEmailChange,  
    label = { Text("Email Anda") },  
    leadingIcon = {  
        Icon(  
            painter = painterResource(id = R.drawable.mail_icon),  
            contentDescription = "Email"  
        )  
    },  
    isError = email.isNotEmpty() && !isEmailValid,  
    supportingText = { if (email.isNotEmpty() && !isEmailValid) Text("Format Email Salah") },  
    modifier = Modifier.fillMaxWidth(),  
    shape = MaterialTheme.shapes.medium  
)
```

9. Dibawah `OutlinedTextField` untuk email dan spacer, tambahkan Component `DropDownMenuBox` beserta deklarasi variabel pendukung seperti gambar berikut:

```
var expanded by remember { mutableStateOf( value = false) }
val options = listOf("Pertanyaan", "Keluhan", "Saran")

ExposedDropDownMenuBox(
    expanded = expanded,
    onExpandedChange = { expanded = !expanded }
) {
    OutlinedTextField(
        readOnly = true,
        value = problemType,
        onValueChange = { },
        label = { Text("Tipe Pesan") },
        trailingIcon = { ExposedDropDownMenuDefaults.TrailingIcon(expanded = expanded) },
        colors = ExposedDropDownMenuDefaults.outlinedTextFieldColors(),
        modifier = Modifier.menuAnchor().fillMaxWidth()
    )
    ExposedDropDownMenu(
        expanded = expanded,
        onDismissRequest = { expanded = false }
    ) {
        options.forEach { selectionOption ->
            DropdownMenuItem(
                text = { Text(text = selectionOption) },
                onClick = {
                    onProblemTypeChange(selectionOption)
                    expanded = false
                }
            )
        }
    }
}
```

Gambar 10. Pembuatan Components berbentuk DropDown

Penjelasan kode:

Secara ringkas, kode pada gambar 10 berfungsi untuk membangun komponen **dropdown menu**, dengan rincian sebagai berikut:

- **Penyimpan status dan opsi data:** Variabel `expanded` berfungsi sebagai penentu apakah daftar menu sedang terbuka atau tertutup (`true/false`), sedangkan `options` menyimpan daftar pilihan yang bisa diklik.
- **Kolom `OutlinedTextField`:** Dibungkus oleh `ExposedDropDownMenuBox` dan bernilai `readOnly = true` agar pengguna tidak mengetik manual, melainkan hanya melihat nilai yang terpilih

(problemType). Modifier .menuAnchor() berfungsi mengaitkan posisi kolom teks sebagai dasar tempat munculnya daftar opsi.

- **Daftar opsi (*ExposedDropdownMenu*):** Menampilkan daftar pilihan melalui perulangan `options.forEach`. Ketika salah satu opsi diklik, aplikasi memicu fungsi *callback* `onProblemTypeChange` untuk mengirim data opsi yang dipilih ke komponen induk, lalu otomatis menutup kembali menunya lewat perintah `expanded = false`.

10. Tambahkan pula komponen untuk menampilkan Uri gambar yang dipilih serta checkbox seperti pada gambar berikut:

```
if (imageUri != null) {
    Spacer(modifier = Modifier.height(8.dp))
    Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
        Row(modifier = Modifier.padding(all = 12.dp), verticalAlignment = Alignment.CenterVertically) {
            Icon(painterResource(id=R.drawable.icon_check), contentDescription = "File")
            Spacer(modifier = Modifier.width(8.dp))
            Text("File terpilih: ${imageUri.lastPathSegment}")
        }
    }
    Spacer(modifier = Modifier.height(12.dp))

    Row(modifier = Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically) {
        Checkbox(checked = isAgreed, onCheckedChange = onAgreedChange)
        Text("Saya menyetujui syarat & ketentuan")
    }
    Spacer(modifier = Modifier.height(16.dp))
}
```

Gambar 11. Implementasi Kode untuk menampilkan Uri gambar serta Checkbox

Penjelasan kode:

- **Tampilan konfirmasi gambar secara kondisional** (`if (imageUri != null)`): Blok tersebut mengimplementasikan logika percabangan kondisional secara deklaratif yaitu komponen `Card` hanya akan dirender ke layar jika variabel `imageUri` memiliki nilai (tidak `null`). Di dalamnya, sebuah `Row` yang berfungsi untuk menyusun `Icon` berdampingan secara horizontal dengan teks nama file yang diambil dari `imageUri.lastPathSegment`.
- **Persetujuan (Checkbox):** Disusun sejajar secara horizontal bersama label teks menggunakan `Row`. Komponen `Checkbox` ini menerapkan pola *stateless*:

status centang dibaca langsung dari variabel `isAgreed`, dan setiap kali kotak disentuh oleh pengguna, perubahan statusnya diteruskan ke komponen induk melalui fungsi *callback* `onAgreedChange`.

11. Lakukan modifikasi button submit dengan mengganti nilai `onClick` seperti gambar berikut:

```
Button(  
    onClick = onSubmit,  
    enabled = isValid,  
    modifier = Modifier.fillMaxWidth(),  
    shape = MaterialTheme.shapes.large  
) {  
    Row(  
        verticalAlignment = Alignment.CenterVertically,  
        horizontalArrangement = Arrangement.Center  
    ) {  
        Icon(  
            painter = painterResource(id = R.drawable.send_icon),  
            contentDescription = "Send"  
        )  
        Spacer(modifier = Modifier.padding(horizontal = 4.dp))  
        Text("Kirim Pesan", style = MaterialTheme.typography.labelLarge)  
    }  
}
```

Gambar 12. Modifikasi Tombol Submit

12. Lakukan pemanggilan *function* `StatelessFormHubungiKami()` pada *function* `HubungiKamiScreen()`

```
) { paddingValues ->  
    StatelessFormHubungiKami(  
        modifier = Modifier.padding(paddingValues),  
        email = emailText,  
        onEmailChange = { emailText = it },  
        isValid = isValid,  
        message = messageText,  
        onMessageChange = { messageText = it },  
        isValid = isValid,  
        problemType = problemType,  
        onProblemTypeChange = { problemType = it },  
        isAgreed = isAgreed,  
        onAgreedChange = { isAgreed = it },  
        imageUri = imageUri,  
        onImagePicked = { imageUri = it },  
        isValid = isValid,  
        onSubmit = {  
            scope.launch {  
                snackbarHostState.showSnackbar( message = "Pesan Terkirim!")  
            }  
        }  
    )  
}
```

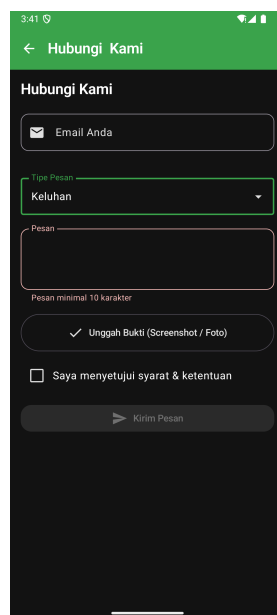
Gambar 13. Pemanggilan Function `StatelessFormHubungiKami`

Penjelasan kode:

Proses tersebut merupakan implementasi/penerapan dari konsep **State Hoisting** dan **Unidirectional Data Flow (UDF)** pada Jetpack Compose, yaitu komponen *parent* mengendalikan atas seluruh status data (seperti `emailText`, `messageText`, dan `isAgreed`) lalu meneruskannya ke bawah (*downward*) ke dalam parameter `StatelessFormHubungiKami`, sekaligus menyalurkan hasil validasi (`isEmailValid`, `isFormValid`) agar tampilan layar selalu sinkron dengan kondisi data terkini.

Ketika pengguna berinteraksi dengan tampilan tersebut, misalnya mengetik email atau memilih gambar, fungsi *callback* seperti `onEmailChange = { emailText = it }` dan `onImagePicked = { imageUri = it }` akan dipicu untuk memperbarui variabel penampung status lokal, yang secara otomatis memicu proses *recomposition*. Bagian akhir pada parameter `onSubmit` mengaitkan konsep **Kotlin Coroutines** melalui fungsi pembangun `scope.launch`; blok asinkron tersebut memastikan operasi penundaan (*suspending function*) seperti `snackbarHostState.showSnackBar` dapat berjalan di latar belakang tanpa memblokir alur utama antarmuka pengguna (*main UI thread*).

Lakukan pratinjau (*preview*) sehingga hasilnya seperti gambar di bawah ini:

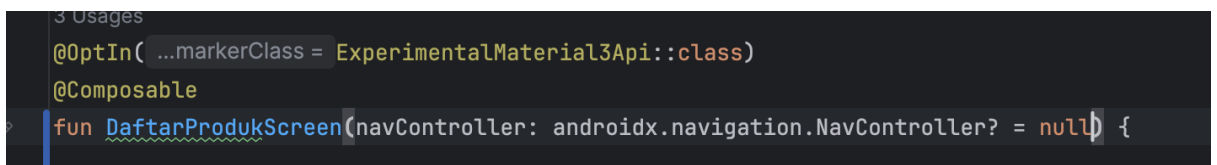


Gambar 14. Hasil Implementasi Form Hubungi Kami

## C. Modifikasi kode pada `DaftarProductScreen.kt`

Pada halaman daftar produk, kita akan menambahkan fitur *Search Bar* (kolom pencarian) yang diintegrasikan dengan filter kategori, serta melakukan simulasi proses asinkronus seolah-olah aplikasi sedang mengambil data dari internet. Pada bagian ini, penjelasan akan lebih sedikit dibandingkan dengan bagian sebelumnya, dikarenakan pola dan konsep kode yang memiliki kesamaan. Silakan ikuti langkah – langkah berikut

1. Tambahkan parameter pada function `DaftarProdukScreen`:



```
3 Usages
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun DaftarProdukScreen(navController: androidx.navigation.NavController? = null) {
```

Gambar 15. Penambahan Parameter pada function `DaftarProdukScreen`

**NavController** adalah objek pengendali utama dalam pustaka Jetpack Navigation di Android. Ibarat nahkoda sebuah kapal, NavController bertanggung jawab atas seluruh pergerakan antarlayar di dalam aplikasi. Objek ini bertugas:

- **Mengatur perpindahan layar:** mengeksekusi perintah navigasi halaman
- **Mengelola tumpukan riwayat halaman (*back stack*):** mencatat urutan layar yang pernah dibuka sehingga saat pengguna menekan tombol kembali (*back button*), aplikasi akan harus kembali ke layar yang mana.
- **Mengirim dan membaca *argument/data*:** membawa data saat berpindah dari satu layar ke layar lainnya.

## 2. Deklarasi Variabel

Lakukan modifikasi variabel pada function DaftarProdukScreen sehingga diperoleh hasil akhir sebagai berikut:



```
fun DaftarProdukScreen() {  
    val context = LocalContext.current  
    var selectedCategoryId by rememberSaveable { mutableStateOf( value = DummyData.categories.firstOrNull()?.id) }  
    var searchQuery by rememberSaveable { mutableStateOf( value = "" ) }  
    var isLoading by remember { mutableStateOf( value = false ) }  
    var filteredProducts by remember { mutableStateOf( value = emptyList<Product>() ) }  
  
    LaunchedEffect( key1 = selectedCategoryId, key2 = searchQuery ) {  
        isLoading = true  
  
        delay( timeMillis = 1000 )  
  
        val filteredByCategory = if (selectedCategoryId != null) {  
            DummyData.products.filter { it.category_id == selectedCategoryId }  
        } else DummyData.products  
  
        filteredProducts = if (searchQuery.isBlank()) {  
            filteredByCategory  
        } else {  
            filteredByCategory.filter { it.name.contains( other = searchQuery, ignoreCase = true ) }  
        }  
  
        isLoading = false  
    }  
}
```

Gambar 16. Implementasi LaunchedEffect untuk proses asinkron

Konsep **asinkron (asynchronous)** dalam pemrograman aplikasi adalah metode eksekusi di mana suatu tugas yang membutuhkan waktu, misalnya: mengunduh data, membaca penyimpanan, atau menunggu waktu karena terdapat proses di latar belakang tanpa menghentikan alur kerja utama program.

Pada sistem operasi Android, alur utama (*main thread*) memiliki tanggung jawab krusial untuk menggambar visual antarmuka dan merespons interaksi sentuhan pengguna, jika tugas berat dijalankan secara sinkron (menunggu hingga selesai), layar ponsel berpotensi membeku (*freeze*). Melalui pendekatan asinkron, aplikasi tetap dapat bergerak mulus dan interaktif selagi proses yang memakan waktu tersebut berjalan mandiri di belakang layar.

LaunchedEffect berfungsi pengendali proses asinkron di dalam komponen Jetpack Compose. Sedangkan perintah `delay(1000)` adalah sebuah fungsi penunda waktu asinkron (*suspending function*) milik Kotlin Coroutines yang menangguhkan eksekusi kode selama 1.000 milidetik (satu detik). Sehingga

fungsi secara keseluruhan kode tersebut adalah selama durasi satu detik, alur kerja di dalam `LaunchedEffect` berhenti sejenak untuk memberi waktu animasi *loading* berputar atau menunggu pengguna selesai mengetik tanpa menggunakan seluruh sumber daya komputasi dan tanpa membuat antarmuka ponsel mengalami *freeze*.

### 3. Membuat Function `StatelessDaftarProduct()`

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun StatelessDaftarProduct(
    categories: List<Category>,
    selectedCategoryId: Int?,
    onCategorySelected: (Int) -> Unit,
    searchQuery: String,
    onSearchQueryChange: (String) -> Unit,
    isLoading: Boolean,
    products: List<Product>,
    onProductClick: (Product) -> Unit
) {
    |
}
```

Gambar 17. Membuat Function `StatelessDaftarProduct()`

### 4. Pindahkan Scaffold beserta seluruh childnya dari function `DaftarProdukScreen` ke `StatelessDaftarProduct()`

```
> Scaffold(...) { paddingValues ->
>     Column(...) {
>
>         OutlinedTextField(...)
>
>         Text(...)
>
>         LazyRow(...) {
>             items(categories) {...}
>         }
>
>         Spacer(modifier = Modifier.height(16.dp))
>         Text(...)
```

Gambar 18. Memindahkan Scaffold dan seluruh child-nya

5. Tambahkan komponen `OutlinedTextField` ke di atas Text “Kategori Produk” seperti pada gambar berikut:

```
Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(paddingValues)
) {

    OutlinedTextField(
        value = searchQuery,
        onValueChange = onSearchQueryChange,
        label = { Text("Cari produk...") },
        modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 8.dp),
        singleLine = true
    )

    Text(
        text = "Kategori Produk",
        style = MaterialTheme.typography.titleLarge,
        modifier = Modifier.padding(all = 16.dp)
    )
}
```

Gambar 19. Implementasi `OutlinedTextField`

6. Ganti parameter `items` pada `LazyRow` seperti gambar di bawah ini:

```
LazyRow(
    contentPadding = PaddingValues(horizontal = 16.dp),
    horizontalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(DummyData.categories) { category ->
        CategoryItem(
            category = category,
            isSelected = category.id == selectedCategoryId,
            onClick = { onCategorySelected(category.id) }
        )
    }
}
```

Gambar 20. Penggantian Parameter

7. Di bawah text “Daftar Produk” tambahkan kode seperti gambar di bawah ini:

```
if (isLoading) {
    Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            CircularProgressIndicator()
            Spacer(modifier = Modifier.height(8.dp))
            Text("Mencari data...")
        }
    }
} else {
    if (products.isEmpty()) {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            Text("Produk tidak ditemukan.")
        }
    } else {
```

Gambar 21. Implementasi blok pengkondisian tampilan item

8. Pindahkan komponen LazyVerticalGrid ke dalam blok else:

```
} else {
    LazyVerticalGrid(
        columns = GridCells.Fixed( count = 2),
        contentPadding = PaddingValues( all = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        modifier = Modifier.fillMaxSize()
    ) {
        items(filteredProducts) { product ->
            ProductItemCard(product = product) {
                Toast.makeText(
                    context, text = "Clicked: ${product.name}",
                    duration = Toast.LENGTH_SHORT
                )
                    .show()
            }
        }
    }
}
```

Gambar 22. Memindahkan Komponen LazyVerticalGrid

9. Ganti Toast serta parameter `items` menjadi seperti di gambar berikut:

```
items(products) { product ->
    ProductItemCard(product = product) {
        onProductClick(product)
    }
}
```

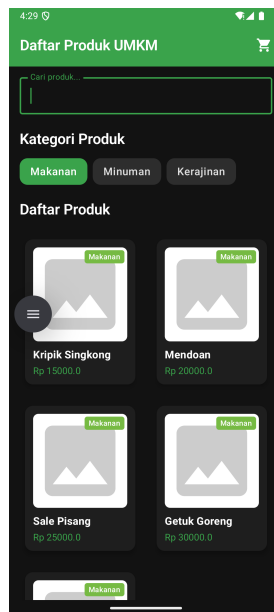
Gambar 23. Penyesuaian Kode pada blok Items

10. Panggil Function tersebut dari function `DaftarProdukScreen()`

```
StatelessDaftarProduct(
    categories = DummyData.categories,
    selectedCategoryId = selectedCategoryId,
    onCategorySelected = { selectedCategoryId = it },
    searchQuery = searchQuery,
    onSearchQueryChange = { searchQuery = it },
    isLoading = isLoading,
    products = filteredProducts,
    onProductClick = { product ->
        navController?.navigate( route = "detail/${product.id}")
    },
    onContactUsClick = {
        navController?.navigate( route = "hubungi_kami")
    }
)
```

Gambar 24. Pemanggilan Function `DaftarProdukScreen()`

11. Cek di-preview, dan jalankan masing masing fiturnya, sehingga diperoleh hasil sebagai berikut:



Gambar 25. Hasil Tampilan Modifikasi Daftar Produk

## D. Membuat Tampilan Detail Produk

1. Buatlah Kotlin file baru dengan nama `DetailProductScreen.kt` kemudian deklarasikan function dan variabel seperti gambar di bawah ini:

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun DetailProductScreen(productId: Int, navController: NavController?) {
    val context = LocalContext.current
    var isLoading by remember { mutableStateOf( value = true) }
    var product by remember { mutableStateOf<Product?>( value = null) }
    var quantity by rememberSaveable { mutableStateOf( value = 1) }

    LaunchedEffect( key1 = productId) {
        isLoading = true
        delay( timeMillis = 1000) // Simulasi loading server lambat
        product = DummyData.products.find { it.id == productId }
        isLoading = false
    }
}
```

Gambar 26. Deklarasi Function `DetailProductScreen` dan Variabel

2. Buat function baru bernama `StatelessDetailProduct` dengan detail parameter seperti pada gambar berikut:

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun StatelessDetailProduct(
    product: Product?, isLoading: Boolean, quantity: Int,
    onQuantityChange: (Int) -> Unit, onBackClick: () -> Unit, onAddToCartClick: () -> Unit
) {
}
```

Gambar 27. Deklarasi Function `StatelessDetailProduct`

3. Di dalam function `StatelessDetailProduct`, tambahkan Scaffold yang berisi logika IF.

```
Scaffold(
    topBar = {
        TopAppBar(
            title = { Text("Detail Produk") },
            navigationIcon = {
                IconButton(onClick = onBackClick) {
                    Icon(painterResource(id = R.drawable.back_icon), contentDescription = "Back")
                }
            }
        )
    }
) { paddingValues ->
    if (isLoading) {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
    } else if (product != null) {
```

Gambar 28. Implementasi Scaffold dan Logika IF

4. Di blok `else-if` tambahkan kolom serta deklarasi komponen Image seperti gambar berikut:

```
} else if (product != null) {
    Column(modifier = Modifier.fillMaxSize().padding(paddingValues).verticalScroll(state = rememberScrollState())) {
        val imageRes = if (product.img == "dummy_product") R.drawable.dummy_product else R.drawable.dummy_product
        Image(painterResource(id = imageRes), contentDescription = null, modifier = Modifier.fillMaxWidth().height(280.dp))
    }
}
```

Gambar 29. Implementasi Kode pada Blok Else-IF

5. Masih di dalam Column, tambahkan lagi komponen *column* seperti gambar berikut dan isikan beberapa komponen text:

```
else if (product != null) {  
    Column(modifier = Modifier.fillMaxSize().padding(paddingValues).verticalScroll( state = rememberScrollState())) {  
        val imageRes = if (product.img == "dummy_product") R.drawable.dummy_product else R.drawable.dummy_product  
        Image(painterResource(id = imageRes), contentDescription = null, modifier = Modifier.fillMaxWidth().height(280.dp))  
        Column(modifier = Modifier.padding( all = 16.dp)) {  
            Text(product.name, style = MaterialTheme.typography.headlineSmall, fontWeight = FontWeight.Bold)  
            Text("Rp ${product.price}", style = MaterialTheme.typography.titleLarge)  
            Spacer(modifier = Modifier.height(16.dp))  
            Text("Deskripsi", fontWeight = FontWeight.Bold)  
            Text(product.description ?: "")  
            Text("Stok: ${product.stock}")  
        }  
    }  
}
```

Gambar 30. Implementasi Kode pada Blok Else-If

6. Di bawah komponen text tambahkan lagi komponen berikut:

```
Spacer(modifier = Modifier.height(24.dp))  
Row(  
    modifier = Modifier.fillMaxWidth(),  
    horizontalArrangement = Arrangement.SpaceBetween,  
    verticalAlignment = Alignment.CenterVertically  
) {  
    Text("Jumlah Beli")  
    Row(verticalAlignment = Alignment.CenterVertically) {  
        FilledTonalIconButton(  
            onClick = { if (quantity > 1) onQuantityChange(quantity - 1) },  
            enabled = quantity > 1  
        ) { Text("-") }  
  
        Text(quantity.toString(), modifier = Modifier.padding(horizontal = 16.dp))  
  
        FilledTonalIconButton(  
            onClick = { if (quantity < product.stock) onQuantityChange(quantity + 1) },  
            enabled = quantity < product.stock  
        ) { Text("+") }  
    }  
}  
Spacer(modifier = Modifier.height(16.dp))  
Button(  
    onClick = onAddToCartClick,  
    modifier = Modifier.fillMaxWidth().height(50.dp),  
    enabled = product.stock > 0 && quantity > 0  
) {  
    Text("Tambah ke Keranjang")  
}
```

Gambar 31. Lanjutan Implementasi Kode pada Blok Else-If

7. Di dalam function `DetailProductScreen`, panggil function `StatelessDetailProduct`

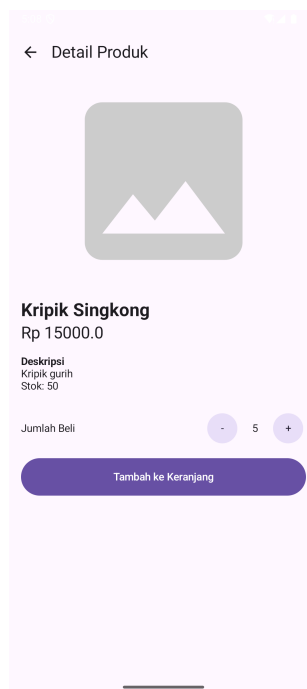
```
fun DetailProductScreen(productId: Int, navController: NavController?) {
    val context = LocalContext.current
    var isLoading by remember { mutableStateOf( value = true) }
    var product by remember { mutableStateOf<Product?>( value = null) }
    var quantity by rememberSaveable { mutableStateOf( value = 1) }

    LaunchedEffect( key1 = productId ) {
        isLoading = true
        delay( timeMillis = 1000 ) // Simulasi loading server lambat
        product = DummyData.products.find { it.id == productId }
        isLoading = false
    }

    StatelessDetailProduct(
        product = product,
        isLoading = isLoading,
        quantity = quantity,
        onQuantityChange = { quantity = it },
        onBackClick = { navController?.popBackStack() },
        onAddToCartClick = { Toast.makeText(context, text = "Dimasukkan: $quantity", duration = Toast.LENGTH_SHORT).show() }
    )
}
```

Gambar 32. Pemanggilan Function

8. Jalankan previewnya dan diperoleh hasil sebagai berikut:



Gambar 33. Hasil Preview Detail Produk

## E. Navigasi di homeactivity untuk daftar produk ke detail produk

Pada bagian ini, kita akan mempraktikkan untuk proses navigasi antar halaman.

1. Buka HomeActivity.kt dan tuliskan kode berikut:

```
class HomeActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            JualanTheme {
                val navController = rememberNavController()
                NavHost(navController = navController, startDestination = "daftar_produk") {
                    composable(route = "daftar_produk") {
                        DaftarProdukScreen(navController = navController)
                    }
                    composable(
                        route = "detail/{productId}",
                        arguments = listOf(navArgument(name = "productId") {
                            type = NavType.IntType
                        })
                    ) { backStackEntry ->
                        val productId = backStackEntry.arguments?.getInt(key = "productId") ?: 0
                        DetailProductScreen(
                            productId = productId,
                            navController = navController
                        )
                    }
                }
            }
        }
    }
}
```

Gambar 34. Modifikasi Kode pada HomeActivity

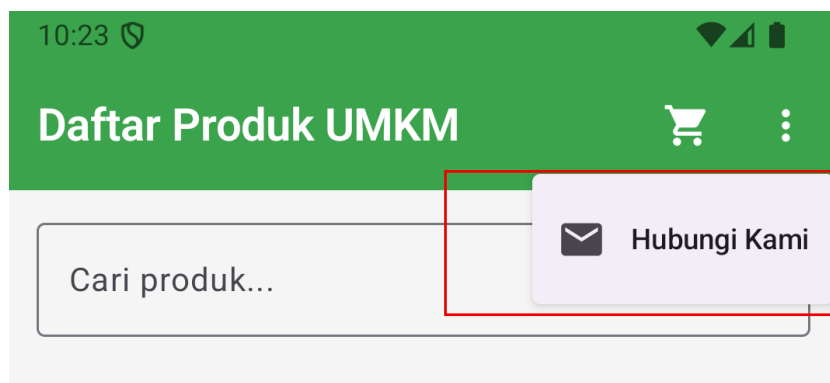
Potongan kode ini berfungsi sebagai **peta rute navigasi** untuk mengatur perpindahan antarlayar di dalam aplikasi Android menggunakan Jetpack Compose. Baris pertama menyiapkan variabel `navController` melalui fungsi `rememberNavController()` sebagai pengendali utama yang bertugas mengarahkan pengguna berpindah layar dan mencatat riwayat halaman yang telah dibuka. Komponen `NavHost` kemudian bertindak sebagai wadah atau terminal utamanya, dengan mendeklarasikan `startDestination = "daftar_produk"`, yang berarti layar pertama yang langsung muncul saat aplikasi dibuka adalah halaman daftar produk.

Di dalam `NavHost`, didefinisikan tiga tujuan rute menggunakan fungsi `composable`. Rute pertama menampilkan halaman utama **DaftarProdukScreen**, sedangkan rute ketiga menampilkan halaman **HubungiKamiScreen**. Berbeda

pada rute kedua, yaitu "detail/{productId}", yang dirancang khusus untuk menerima kiriman data angka berupa ID produk (NavType.IntType). Ketika pengguna memilih produk tertentu, sistem membaca ID tersebut lewat backStackEntry, mengambil angkanya (atau memberi nilai bawaan 0 jika kosong), lalu meneruskannya ke layar DetailProductScreen agar halaman detail dapat menampilkan data produk yang sesuai secara dinamis.

## F. Menambahkan Menu Action di AppBar untuk navigasi ke hubungi kami

Silakan buka Kembali file: DaftarProdukScreen.kt. Pada bagian ini kita akan mempraktikkan untuk menambahkan MenuAction seperti pada gambar di bawah ini:



Gambar 35. Menampilkan Menu Action

1. Tambahkan 1 parameter yang bernama onContactUsClick()

```
fun StatelessDaftarProduct(  
    categories: List<Category>,  
    selectedCategoryId: Int?,  
    onCategorySelected: (Int) -> Unit,  
    searchQuery: String,  
    onSearchQueryChange: (String) -> Unit,  
    isLoading: Boolean,  
    products: List<Product>,  
    onProductClick: (Product) -> Unit,  
    onContactUsClick: () -> Unit  
) {
```

Gambar 36. Penambahan Parameter onContactUsClick()

2. Pada pemanggilan function, tambahkan parameter seperti pada gambar berikut:

```
StatelessDaftarProduct(  
    categories = DummyData.categories,  
    selectedCategoryId = selectedCategoryId,  
    onCategorySelected = { selectedCategoryId = it },  
    searchQuery = searchQuery,  
    onSearchQueryChange = { searchQuery = it },  
    isLoading = isLoading,  
    products = filteredProducts,  
    onProductClick = { product ->  
        navController?.navigate(route = "detail/${product.id}")  
    },  
    onContactUsClick = {  
        navController?.navigate(route = "hubungi_kami")  
    }  
)
```

Gambar 37. Penambahan Parameter onContactUsClick()

3. Tambahkan kode `IconButton` di bawah kode ikon keranjang serta tambahkan pula dropdown seperti pada gambar 38.

```
var expanded by remember { mutableStateOf( value = false) }

IconButton(onClick = { expanded = true }) {
    Icon(
        imageVector = Icons.Default.MoreVert,
        contentDescription = "Menu",
        tint = MaterialTheme.colorScheme.onPrimary
    )
}

DropdownMenu(
    expanded = expanded,
    onDismissRequest = { expanded = false }
) {
    DropdownMenuItem(
        text = { Text("Hubungi Kami") },
        onClick = {
            expanded = false
            onContactUsClick()
        },
        leadingIcon = {
            Icon(
                imageVector = Icons.Default.Email,
                contentDescription = "Email"
            )
        }
    )
}
```

Gambar 38. Penambahan `IconButton`

**JALANKAN APLIKASI TERSEBUT DI *EMULATOR*/PERANGKAT ANDA**